

Resource Bounded Program Equilibria under Cryptographic Commitments and Revocable Authority

AI Safety: Safeguarding Multi-Agent Interactions through Deterministic State Controllers

Dr. Carsten Stöcker, Spherity GmbH

28 September 2026



Except where otherwise noted, this work is licensed under the Creative Commons Attribution 4.0 International License (CC BY 4.0). © 2026 Dr. Carsten Stöcker, Spherity GmbH.

Abstract

Program games allow agents to condition decisions on counterpart programs, creating opportunities for cooperation. Their deployment in AI systems requires accounting for verification costs, revocable authority and the connection between certified decisions and external effects. We analyse finite, resource-bounded program games coupled to a protected deterministic execution controller. We derive an exact unilateral-deviation criterion that separates active-game payoff advantages, completion before revocation and differential implementation costs. Revocation can favour faster deviations even when its time distribution is independent of program selection. A worked compute-booking example shows how an authorised individual fallback can become preferable to a higher-value joint booking. To connect certified proposals to digital effects, we specify an epoch-bound controller and prove authority, aggregate-budget, exact-action, delegation-scope and replay properties under sound checking, complete mediation and atomic execution. A shared-budget example shows why two individually affordable requests require joint accounting when their combined commitments exceed a common mandate. The controller invariants hold for arbitrary proposal sequences, including those generated by probabilistic AI agents; strategic guarantees remain relative to the declared game. We also establish a private-interface discrimination bound and a conditional equilibrium-transfer bound that includes implementation costs. Exact rational checks cover 38,880 parameter combinations, and exhaustive exploration of two finite controller abstractions reaches 1,427 and 1,819 states. Five individual control omissions each produce a counterexample. These results inform applied AI safety by separating incentive compatibility, adequate decision evidence and authorised execution, and by identifying how execution safeguards can change strategic incentives. The guarantees concern specified games and mediated digital effects; policy adequacy, physical safety and general alignment require separate arguments.

Keywords: program equilibrium; bounded rationality; deterministic controller; bounded Löb; cryptographic commitments; cryptographic identifiers; revocation; capability security; runtime assurance; AI safety; multi-agent systems; trust infrastructure; delegated authority; verifiable credentials; evidence provenance

Contents

1	Introduction	3
2	Related work and research method	4
2.1	Program games and computational costs	4
2.2	Certified computation and controlled execution	5
2.3	Scope of the literature review	5
3	The bounded program game	6
3.1	Principals programs and resources	6
3.2	Payoffs timing and authority	8
3.3	Authority and the represented principal	8
3.4	Why the baseline assumes determinism	9
4	The incentive effect of cost and revocation	11
4.1	An exact deviation criterion	11
4.2	Common survival probabilities	12
4.3	Optional checking and zero margins	12
4.4	Faster deviations under the same external revocation process	13
4.5	Profile dependent abort payoffs	15
4.6	Mandatory checking with alternative evidence routes	15
5	Bounded proofs and private interfaces	15
5.1	What bounded Löb supplies	15
5.2	A conditional proof length calculation	16
5.3	Strategically relevant information	17
5.4	Historical traces and future action	18
6	An epoch bound action controller	20
6.1	State and trust boundary	20
6.2	Transitions	21
6.3	Invariants	23
6.4	Trace refinement and distributed revocation	24
7	Cryptographic realisation and equilibrium transfer	24
7.1	The certified relation	24
7.2	Cost aware simulation	25
7.3	The transfer result	26
7.4	What a full realisation still requires	27
8	Reproducible finite analysis	27
8.1	Exact utility comparisons	27
8.2	A finite response table game	28
8.3	Exhaustive controller instances	28
8.4	Deliberate control omissions	29
8.5	Reproduction	29
9	Implications for agent assurance	30
9.1	Cooperation is not a safety specification	30
9.2	The immediate engineering boundary	30
9.3	Resulting constraints for probabilistic AI agents	30
9.4	Applicability to multi agent systems	31
9.5	Monitoring evaluation and behavioural change	32
9.6	Execution safety and useful progress	32
10	Limitations and further research	33
A	Research sequence and claim boundaries	34
B	Executable finite checks	35
	References	39

1 Introduction

An agent can present convincing evidence about a program while lacking current authority to execute it. It can also implement a cooperative strategy that ceases to be a best response once verification costs are included. These are different failures. The first concerns the connection between evidence and effects. The second concerns incentives in the implemented game.

Incentives are the differences in expected utility that make one feasible choice more attractive than another. Utility represents the principal’s modelled preferences over outcomes, including the costs of obtaining them. An agent may be permitted to make two bookings yet have an incentive to choose the cheaper or faster one. Authorisation determines what may execute; incentive analysis compares the permitted alternatives and other deviations admitted by the game. Neither concept assumes that an LLM always identifies or follows a best response.

In a program equilibrium, the strategic choice is a decision procedure rather than one isolated action. A procedure can inspect permitted information about another procedure and respond conditionally. Changing one player’s program can therefore change another player’s response even when the latter’s program remains fixed. All deviation payoffs below include those responses. The program-equilibrium formulation has an established foundation in Tennenholtz [1].

Open-source game theory studies agents that can reason about other agents’ procedures. Logical agents can achieve cooperation without requiring byte-for-byte equality of their source code [2]. Bounded variants establish cooperation under finite proof-length conditions [3, 4]. Simulation-based approaches provide another route to program equilibria, with results that depend on the available randomness and program class [5]. These results motivate examining what survives when a mathematical agent becomes an authorised actor in a running system.

We focus on a specific setting. A finite set of principals selects bounded programs. Each program proposes an action or an abort outcome. A protected controller checks the action against current authority and commits the covered digital effect. Program descriptions or selected properties may be cryptographically committed. A principal can revoke authority through an authenticated channel outside the proposing program’s control.

Running example. Two engineering organisations use agents to arrange a joint booking of compute capacity. Cooperation means following the specified joint-booking arrangement. A controlled service also allows an individual fallback booking. Each organisation may use several worker agents under one spending mandate. The example’s terminal outcome is an atomic booking record and its budget charge, not completion of the later simulation. This keeps the illustration within the paper’s all-or-abort digital model. Joint value, verification effort, private inputs and revocable booking rights provide concrete counterparts for the formal quantities.

Game cooperation and AI safety. A Nash equilibrium excludes profitable unilateral program changes in the declared game. It does not prevent a player from deviating, establish resistance to coalitions, or ensure that the outcome protects outsiders. The controller enforces the constraints encoded in its specification. Whether those constraints address the relevant harms is a separate validation question. Thus game cooperation, execution conformance and safety adequacy require distinct arguments.

Three questions organise the analysis. First, which cooperative profiles survive implementation costs and revocation? Second, which information must a private interface preserve for a counterparty to discriminate between strategically different programs? Third, which operational invariants connect the certified decision to its execution?

Our first result gives an exact incentive margin for a timed implementation. It distinguishes an independently sampled revocation time from a strategy-independent probability of successful

completion. This distinction matters because faster programs can complete in a larger fraction of the same authority windows. Our second set of results identifies conditions for transporting proof witnesses and limits on discrimination through private interfaces. Our third result is an inductive safety argument for a controller with current epochs, exact action binding, replay protection and shared reservations. A final transfer theorem connects an ideal, revocation-aware game to a concrete implementation when deviations admit a cost-aware simulation.

The contribution is a common formal account of three deployment obligations: retaining the intended incentive margin, exposing the information needed for strategic discrimination, and enforcing current authority and resource constraints at the covered effect boundary. Costly computation, secure implementation and proof-carrying execution each have substantial prior art. We therefore identify the elementary or standard parts of the arguments explicitly. We make no claim to a new bounded Löb theorem, a new general folk theorem, or a new cryptographic proof system.

This is the first paper in a three-part series. Paper 2, Risk-Bounded Runtime Assurance for Dynamic Multi-Agent Systems under Partial Observation, adds conditional continuation assurance, joint dependencies and persistent episode accounting [6]. Paper 3, Private Verification of Action Eligibility across Trust Domains, specifies how another organisation can verify bounded claims and bind a decision to current admission and subsequent effects [7]. The present paper supplies the incentive and authority foundation. The later papers add distinct premises; their probability and privacy claims do not follow from the controller invariants alone.

Reader guide. For the concepts and practical interpretation, read Section 3, the worked examples in Sections 4.4 and 4.6, and Sections 6 and 9. For the formal arguments, follow Theorems 1–4 and their adjacent interpretations and conditions for use. Section 8 and the appendices separate finite checking from the general mathematical claims. Figures 1–4 connect the execution architecture, incentive calculation, private-interface limit and controller guards.

2 Related work and research method

2.1 Program games and computational costs

Tennenholtz formalises strategic choice over programs that can inspect counterpart programs [1]. Bárász et al. use provability logic to construct agents whose cooperation depends on semantic reasoning about counterpart programs [2]. Critch develops a resource-bounded logical foundation [3]. Critch, Dennis and Russell analyse several institution designs and identify unresolved questions about bounded interactions and practical proof search [4]. Cooper, Oesterheld and Conitzer characterise simulation-based constructions with and without shared randomness [5].

Computation costs are an established part of the subject. Fortnow discounts utility by computation time and derives program-equilibrium results in that model [8]. Halpern and Pass place computation inside players’ utility functions and relate cost-aware game-theoretic implementation to cryptographic security [9]. Their work is especially important here: preserving an output distribution is insufficient when an implementation changes the cost of producing it.

Conditional disclosure also predates this study. DiGiovanni and Clifton construct commitment games with private information and relate them to approximate Bayesian program equilibria [10]. Kalai et al. analyse voluntary conditional commitment devices in a general commitment-game framework [11]. We examine a narrower deployment question in which authority has an explicit lifecycle and covered effects must pass through a current-state controller.

2.2 Certified computation and controlled execution

Proof-carrying code couples executable code with checkable evidence for a declared safety policy [12]. Proof-carrying data extends related ideas to distributed computations [13]. Complete mediation and small protection mechanisms are established security principles [14]. Recent certified-agent proposals separate action generation, certification and execution [15, 16]. In particular, the realised-trace theorem in Liu Yanglet et al. assumes execution conformance that preserves permissibility [15, Theorem 2]. These proposals use different evidence models, including formal trace permissibility and runtime receipts; their guarantees should be compared at the level of the relation checked and the execution mechanism. Our controller analysis makes selected operational guards explicit. It does not establish conformance for arbitrary runtimes.

Intervention raises a separate incentive question. The Off-Switch Game studies an agent’s incentives to permit human intervention [17]. Our baseline assumes an authenticated intervention mechanism that the agent cannot disable. Strategic interference with that mechanism therefore lies outside the baseline and must be represented as an additional deviation before the results can apply.

Linearizability supplies the relevant ordering concept for concurrent controller operations [18].

The distinction between enforcing a safety property and ensuring eventual progress also has a substantial execution-monitoring literature [19]. Here, an invariant constrains every reachable state or covered action; it does not establish that a useful booking eventually completes. Safety specifications also require an adequate model of the effects being constrained [20]. Attestation freshness and credential verification supply useful infrastructure, but neither establishes all assumptions of the game or the factual truth of a safety claim [21, 22].

2.3 Scope of the literature review

The review used the open-source game theory reading list [23] as a starting point, followed citations to primary work on computation costs, conditional disclosure and implementation, and examined recent certified-agent proposals. The targeted source review was updated on 23 September 2026. The review is targeted rather than a systematic census of the field.

The discussion of progress draws on established execution-monitoring work [19]. Peer-reviewed publications, preprints and technical specifications retain their separate status in the references.

The method has four parts: analytical derivation; counterexample construction; inductive reasoning about a transition system; and exhaustive checks of finite instances. The computational checks use exact rational arithmetic and a breadth-first exploration. They test the specified abstraction. They do not execute a bounded Löbian proof search, benchmark a zero-knowledge virtual machine (zkVM), or evaluate an LLM.

Table 1 identifies the status of the principal results.

Table 1: Results and their evidence

Result	Evidence	Scope
Cost and revocation margin	Exact derivation and rational checks	Specified timed game
Proof witness transport	Conditional length argument	Supplied translated proofs
Private interface limit	Indistinguishability reduction	Complete observation views
Controller safety	Induction and finite exploration	Covered atomic digital effects
Equilibrium transfer	Cost-aware simulation bound	Declared deviation class

3 The bounded program game

3.1 Principals programs and resources

Table 2 fixes the meaning of terms used across the game and the implementation. A principal, a program and a running agent instance are different objects.

Table 2: Concepts used in the analysis

Concept	Meaning in this paper
Principal	Person or organisation whose preferences and authority an agent represents
Program	Procedure mapping permitted observations and state to an action or abort
Program profile	One selected program for every principal, considered together
Agent instance	Particular running process; several instances may act for one principal
Deterministic procedure	Same complete state and explicit inputs yield the same defined computation
Probabilistic procedure	Choices also depend on random inputs with a specified distribution
Nondeterministic model	Several possible transitions without assigned probabilities
Narrow decision kernel	Bounded procedure checking a precise property of a proposal and its evidence
State controller	Protected component that maintains authority and resource state and governs covered effects

The kernel can check whether a proposal satisfies a policy against supplied inputs. The state controller must additionally establish whether that proposal remains authorised and affordable at execution. A deterministic state transition can accept changing external inputs, including authenticated time and revocation events.

Let $N = \{1, \dots, n\}$ be a finite set of principals. Principal i selects a deterministic program p_i from a declared non-empty finite set $\mathcal{P}_i$. Programs have bounded descriptions, execution steps and memory. A timeout maps to a specified abort or default outcome. The resolution procedure

fixes the initial inputs and execution schedule used by this deterministic baseline. For every program profile p , it returns a well-defined terminal outcome. All payoff values are finite real numbers, and expected implementation costs are finite.

The game is defined over program selection. It is not a game in which a controller is assumed to choose a safe outcome on behalf of a player. A program may inspect public descriptions or permitted interface messages. The available information is part of the game definition.

The resource specification distinguishes four quantities:

$$\mathcal{B} = (k, L, \tau, \lambda). \quad (1)$$

Here k is a proof-length allowance, L is a program-description bound, τ is an execution-step or explicitly defined deadline bound, and λ is a cryptographic security parameter. These quantities are not interchangeable. A short proof may be expensive to discover. A succinct cryptographic argument may reduce verification or communication while leaving proof discovery costly.

The memory bound is fixed separately from the tuple in Equation (1). A step count becomes a wall-clock time only through an explicit execution model. The cryptographic parameter is neither a time budget nor permission to spend resources.

Finiteness gives a precise deviation set for the elementary results. Mixed program selection means an external lottery over these finite sets, evaluated by the expectation of the selected programs' utilities. This convention adds no separate selection cost. A pure-profile equilibrium can be tested against pure deviations because utility is linear in that lottery. A randomised program with its own execution costs requires its own strategy and payoff specification. The computational-security extension instead considers uniform families with an explicitly bounded adversary class.

Table 3: Main parameters and quantities

Symbol	Definition	Interpretation
k	Proof-length allowance	How long an admissible formal derivation may be
L	Program-description bound	Maximum permitted description size
τ	Declared execution bound	A step cap or a separately specified deadline; the chosen meaning must be stated
λ	Cryptographic security parameter	Indexes the security and resource assumptions of the cryptographic family
$m_{\max}$	Separate memory bound	Maximum working memory for the declared program class
$u_i(p), a_i$	Active and abort payoffs	Value if the booking completes while authorised, and value on abort
$c_i(p)$	Expected implementation cost	Relevant computation, communication and delay costs in utility units
$t(p), R$	Completion time and revocation time	When the whole profile would complete, and when authority ends
$q(p)$	$\Pr(R > t(p))$	Probability that authority survives until completion

Continued on next page

Table 3 (continued)

Symbol	Definition	Interpretation
$g_i, \Delta c_i, M_i$	Payoff margin, cost saving, net margin	Terms used to test a specific deviation
$e, B, s, \mathcal{R}, U$	Epoch, limit, spent, reservations, used IDs	Current controller state and resource accounting

3.2 Payoffs timing and authority

Write $u_i(p)$ for the active-game payoff of principal i if the selected profile completes while authorised. Let a_i be a common abort payoff for that principal. In the baseline it is independent of the selected profile. Let $t(p)$ be the deterministic time at which the unrevoked resolution procedure returns its terminal outcome, including a specified timeout or default. If that outcome is the common abort, set $u_i(p) = a_i$. Thus resolution completion need not mean that a useful booking succeeds.

An external authority samples a revocation time R independently of program selection. Its survival function is

$$S(t) = \Pr(R > t), \quad q(p) = S(t(p)). \quad (2)$$

A proposed booking succeeds only if the resolution accepts it and authority is still valid at the final commit point. This is an all-or-abort baseline. Workflows with irreversible partial effects require their own intermediate outcomes and payoffs.

Let $c_i(p)$ be expected implementation cost under the authority process. It includes relevant proof search, verification, communication and delay costs expressed in the same utility scale. A cost incurred only on successful completion contributes its expectation; it must not be counted as an unconditional fee as well. The implemented utility is

$$V_i(p) = a_i + q(p)(u_i(p) - a_i) - c_i(p). \quad (3)$$

An external revocation distribution does not make $q(p)$ constant across profiles. Program-dependent completion times can change $q(p)$ even when the law of R is fixed. This is the distinction used in Section 4.

The model also admits a primitive profile-dependent completion probability when a full timing model is unavailable. Such probabilities are then assumptions of the example, not estimates of real agent behaviour. Correlation between execution time, stochastic outcomes and revocation would require taking a joint expectation instead of using Equation (3) in this form.

3.3 Authority and the represented principal

The principal selecting a program, the running instance and the key presenting a certificate are separate objects. A grant identifies its issuer, represented principal, actor, scope, parent grant, per-action resource bounds and validity context. The controller below enforces a cumulative budget at the root mandate. A separate cumulative child-grant cap requires an additional ledger; action-scope inclusion alone does not supply that guarantee. Public identifiers can be implemented in several ways; the results do not depend on a particular decentralised identifier (DID) method or credential syntax.

The controller represents machine-evaluable authority. It does not decide whether an organisation’s internal policy or representation claim has the intended legal effect. Authentic issuers and correctly specified roots of authority are explicit assumptions.

The scope is a set of admissible action records, not an unconstrained purpose label. An action record includes the actor, audience, resource, operation, amount or other parameters, configuration commitment, policy version, epoch and unique action identifier. A natural-language instruction becomes part of the formal model only through a specified interpretation.

3.4 Why the baseline assumes determinism

Deterministic programs and a specified resolution procedure provide a well-defined outcome and unrevoked completion time for each profile. This makes the incentive calculation transparent. Determinism is a modelling choice, not a claim that deterministic programs are cooperative or safe. External revocation remains random in the baseline.

Three sources of variation should be separated. An agent can sample a plan; the environment can change its inputs; and concurrent execution can admit different schedules. A probabilistic model assigns distributions. A nondeterministic transition model can instead require an invariant to hold for every admitted ordering. Fixing a program, its complete inputs and a random tape describes one deterministic run, but does not establish the distribution or quality of other runs.

The state controller can therefore remain deterministic while receiving proposals from a probabilistic AI agent. For the same authoritative state and request, its specified decision is the same. Different concurrent request orders may produce different valid reservations; the invariant must hold for each order. Figure 1 separates this stateful control from proposal generation and optional cryptographic proving.

Proposal, evidence and current execution state

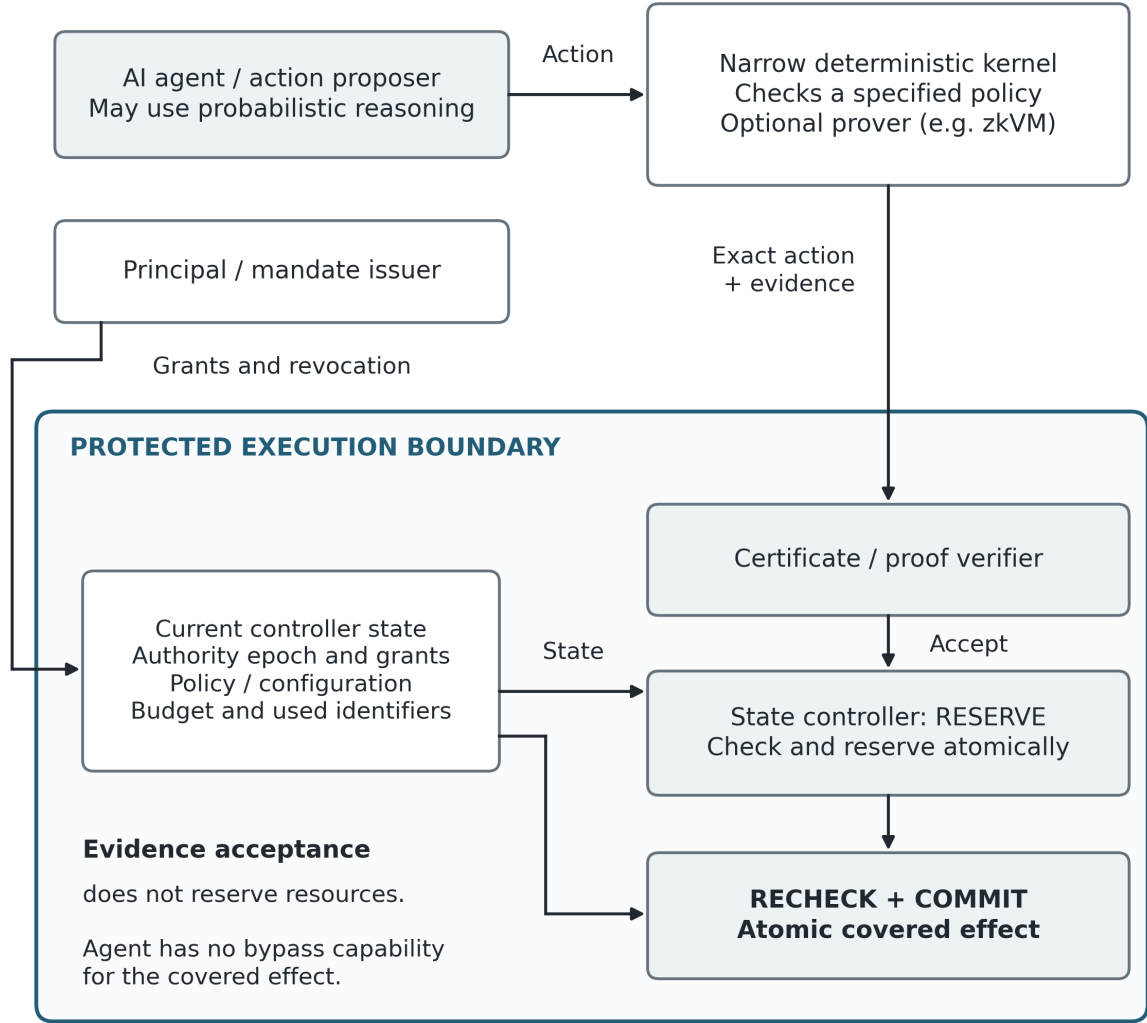


Figure 1: Proposal generation and controlled execution. The AI agent supplies an exact action and evidence. A narrow kernel checks a bounded policy relation; an optional prover supplies a cryptographic argument about that computation. The protected verifier and state controller remain distinct. Authority and resource state are checked at reservation and commit. The atomic covered effect lies inside the assumed execution boundary. The proposer has no bypass capability for that effect. A remote resource outside this boundary needs a separate execution and outcome protocol. This is a proposed architecture, not a benchmarked implementation.

4 The incentive effect of cost and revocation

An incentive is comparative: it exists relative to an alternative. The question is whether principal i obtains greater expected utility by replacing its candidate program while the other principals keep theirs. Such a replacement is a unilateral deviation. It can be entirely authorised and still undermine the intended cooperative arrangement.

Utility need not mean money. In the booking example it represents the value of obtaining suitable capacity, reduced by implementation costs. A best response is a permitted program with maximal expected utility against the other selected programs. A Nash equilibrium is a profile in which every selected program is a best response within the declared deviation class. This is a conditional statement about incentives, not a prediction that a learning agent will discover the profile or that the principals' objectives are appropriate.

The modelled utility is an analytical representation of the principal's preferences. It is distinct from a model's training reward or a natural-language instruction. Whether a deployed agent behaves in accordance with that utility requires separate evidence.

The mechanism determines which incentives arise. Mandatory checks can exclude an uncertified action from execution. Fees and verification effort affect cost. Deadlines and revocation affect the chance of reaching commit. Counterpart programs affect the outcome after a deviation. Changing any of these can change the implemented game. For example, requiring a previously optional proof removes one cheap route, but its cost and timing still have to be included for every route that remains.

The controller's role is to enforce the declared execution constraints. Among actions that pass those constraints, the strategic analysis asks whether the principal prefers the cooperative program or another one. This separation explains why cryptographic soundness cannot, by itself, make cooperation a best response.

4.1 An exact deviation criterion

Fix a candidate profile p^* and a unilateral deviation $d_i \in \mathcal{P}_i$. Abbreviate $p^d = (d_i, p_{-i}^*)$, $q^* = q(p^*)$ and $q^d = q(p^d)$. Define

$$\begin{aligned} g_i(d_i) &= u_i(p^*) - u_i(p^d), \\ \Delta c_i(d_i) &= c_i(p^*) - c_i(p^d). \end{aligned} \tag{4}$$

The quantity g_i is the active-game payoff margin. It can be zero even when the candidate is an active-game equilibrium. A positive Δc_i means the deviation is cheaper.

Intuition. Start with the candidate's payoff advantage when the interaction completes. Weight that advantage by the candidate's completion probability. Then subtract any benefit the deviation gains from more often finishing before revocation, and subtract any implementation cost it saves. The remaining quantity is the incentive margin. The timing term can be negative: more frequent completion is disadvantageous when the deviation's active payoff lies below its abort payoff.

Theorem 1 Exact implementation margin. Under Equation (3), the candidate profile is a Nash equilibrium of the implemented finite game if and only if, for every principal and every permitted unilateral deviation,

$$q^* g_i(d_i) \geq (q^d - q^*) (u_i(p^d) - a_i) + \Delta c_i(d_i). \tag{5}$$

Equivalently, its margin against that deviation is

$$M_i(d_i) = q^\star g_i(d_i) - (q^d - q^\star)(u_i(p^d) - a_i) - \Delta c_i(d_i). \quad (6)$$

Proof. Subtract the deviating utility from the candidate utility:

$$V_i(p^\star) - V_i(p^d) = q^\star(u_i(p^\star) - a_i) - q^d(u_i(p^d) - a_i) - \Delta c_i(d_i).$$

Adding and subtracting $q^\star(u_i(p^d) - a_i)$ gives Equation (6). The candidate is a Nash equilibrium exactly when every such difference is non-negative. If that condition holds for pure deviations, it holds for mixtures by linearity. Conversely, any negative margin supplies a profitable pure deviation. $\square$

The theorem is an exact accounting identity applied to this timed game. Its usefulness is to expose quantities that a certificate-only argument omits. It is not a new general existence theorem.

What this means. A non-negative $M_i(d_i)$ means that the displayed deviation is no better than the candidate. A negative margin identifies an incentive to change program. Equilibrium requires a non-negative margin against every permitted deviation, including cheaper strategies that still cooperate.

Conditions for use. Equation (3) specifies the outcome and cost model. Completion concerns the entire profile, including counterpart responses. Expected costs must include the relevant aborted runs. The criterion concerns the declared program class and unilateral deviations; it supplies neither a coalition guarantee nor an automatic learning or convergence result.

4.2 Common survival probabilities

Corollary 1 Time-neutral authority. If every compared profile has the same completion probability q , the candidate is an equilibrium precisely when $qg_i(d_i) \geq \Delta c_i(d_i)$ for every deviation. If costs are also equal across the compared profiles, every active-game equilibrium is preserved for $q > 0$.

Proof. Substitute $q^d = q^\star = q$ in Equation (5). Equal costs make the right-hand side zero. $\square$

Uniform external fees can remove one source of differential cost, but private proof-search costs may still differ. Equalising a gateway fee therefore does not establish the corollary's full cost premise. Likewise, a common execution slot establishes time neutrality only if every admitted program can meet it and the controller actually delays early execution to that slot.

4.3 Optional checking and zero margins

Corollary 2 Cheaper outcome-equivalent deviation. Suppose a deviation preserves the active payoff and completion probability but reduces expected cost strictly. Then the candidate is not an equilibrium of the implemented game.

Proof. The active margin and timing term vanish. Equation (6) equals the negative of the positive cost saving. $\square$

The condition can arise when a proof-based cooperator faces an opponent that would also cooperate with a cheaper unconditional cooperator. It must be checked for the specified opponent and proof bound; it is not true for every proof-based strategy. For example, a policy that rejects unconditional cooperators changes the response and may remove that particular deviation.

This result separates a cooperation theorem from a best-response argument. Stronger cryptographic soundness does not change a positive saving from optional work. A mechanism that requires the work changes the feasible game and must be analysed as such.

In the running example, a worker may perform an optional comparison of several capacity offers even when a fixed authorised offer would produce the same booking. If the other program's response and completion probability remain unchanged, dropping that extra work saves cost. This is different from omitting a certificate required by the service: the latter request is rejected. The example must therefore specify which work is optional and whether skipping it changes the counterpart's response.

4.4 Faster deviations under the same external revocation process

Suppose a cooperative candidate yields active payoff 3, while a deviation yields 1. Both have abort payoff zero. Let the candidate's completion probability be $3/10$, with expected cost $1/10$. A faster deviation has completion probability one and zero cost. The candidate yields $4/5$, while the deviation yields 1.

The active-game margin is positive, but the implemented margin is $-1/5$. Treating both completion probabilities as $3/10$ would incorrectly produce a positive margin of $1/2$. The difference comes entirely from timing.

These probabilities can be realised by one exogenous revocation law. For example, let R equal 1 with probability $7/10$ and 3 with probability $3/10$. A profile completing at time 2 survives with probability $3/10$; one completing at time $1/2$ survives with probability one. The example makes no claim about a measured revocation rate.

Worked booking interpretation. The candidate arranges the higher-value joint booking. A permitted individual fallback has lower active value, but resolves sooner and avoids the extra coordination work. Both pass the mandatory gateway checks. The costs in Table 4 are additional expected costs after removing any common unconditional upfront fee from both utilities. A fee charged only on successful completion generally cannot be removed this way when completion probabilities differ.

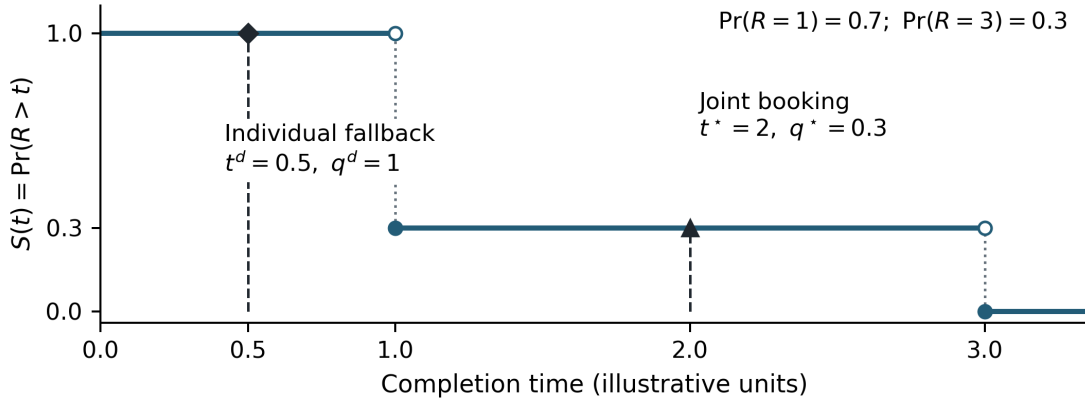
Table 4: Illustrative joint booking and permitted fallback

Quantity	Joint booking profile	Deviating profile with fallback
Active payoff for the deviating principal	3	1
Unrevoked profile completion time	2	0.5
Probability of completing while authorised	0.3	1
Additional expected implementation cost	0.1	0
Implemented expected utility	0.8	1

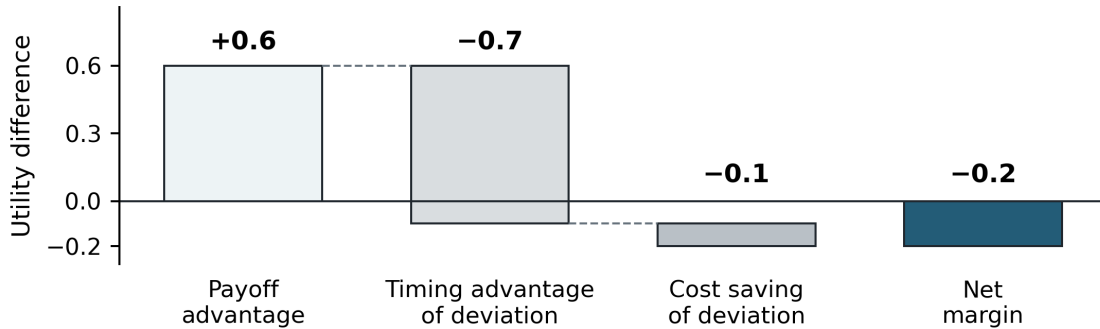
All times and payoffs are illustrative model units. The two completion times describe the complete profiles, including how the unchanged counterpart program responds. Replacing one fast subroutine alone does not establish that the whole interaction finishes sooner.

The active payoff advantage contributes $0.3(3 - 1) = 0.6$. Earlier completion benefits the fallback by $(1 - 0.3)(1 - 0) = 0.7$. Its cost saving is 0.1. Thus the implementation margin is $0.6 - 0.7 - 0.1 = -0.2$. The deviation gains 0.2 utility units. Holding the fallback utility at 1 and the joint-booking cost at 0.1, stability against this fallback requires $3q^* - 0.1 \geq 1$, or $q^* \geq 11/30 \approx 0.367$. This is a threshold for the displayed comparison, not for every possible deviation. Figure 2 shows why a fixed external revocation law produces different survival probabilities.

A One revocation law; different profile completion times



B Incentive margin: joint booking minus individual fallback



$$V_i(p^*) = 0.3 \times 3 - 0.1 = 0.8 < V_i(p^d) = 1$$

Illustrative example. All payoffs and expected costs use one utility scale.

Figure 2: Authority windows and the incentive margin. The same illustrative revocation law applies to both profiles: $R = 1$ with probability 0.7, or $R = 3$ with probability 0.3. The permitted fallback completes at 0.5 and the joint profile at 2. The survival function uses the strict commit convention $R > t$. The margin decomposes into +0.6 from the active payoff advantage, -0.7 from the deviation's timing advantage, and -0.1 from its cost saving. Values are calculated from the stated example, not measured agent performance.

This comparison establishes a profitable deviation from the candidate. It does not establish that all cooperative equilibria disappear. Section 8.2 separately examines the complete deviation set of a small response-table game.

4.5 Profile dependent abort payoffs

If compensation or partial execution makes the abort payoff profile-dependent, write it as $a_i(p)$. Abbreviate $a_i^* = a_i(p^*)$, $a_i^d = a_i(p^d)$, $u_i^d = u_i(p^d)$ and $\Delta q = q^d - q^*$. Suppressing the deviation argument in g_i and Δc_i , direct subtraction gives

$$\begin{aligned} V_i(p^*) - V_i(p^d) &= q^* g_i - \Delta q (u_i^d - a_i^d) \\ &\quad + (1 - q^*) (a_i^* - a_i^d) - \Delta c_i. \end{aligned} \tag{7}$$

The extra term must be retained. Declaring cancellation to be a neutral administrative event is therefore a substantive game assumption. If players can alter the revocation process itself, that ability belongs in the deviation set.

4.6 Mandatory checking with alternative evidence routes

Additional example: two evidence routes for one purchase order. An engineering organisation uses an agent to submit an atomic digital purchase order. The application acceptance profile permits either an authenticated verdict from an agreed evaluator or a private proof that the same bounded eligibility predicate accepts. Both routes perform mandatory checks. The verdict route trusts the evaluator for its computation and permits the required disclosure to that service; the proof route instead assumes sound verification of the specified computation. These are explicit, different trust premises. For this comparison, each is accepted by the same policy and the unchanged counterpart program.

Suppose either complete profile yields active payoff 5, abort payoff 0 and completion probability 0.8. Expected implementation cost is 0.6 for the private-proof route and 0.2 for the accepted-verdict route, including relevant failed attempts. The resulting utilities are $0.8 \times 5 - 0.6 = 3.4$ and $0.8 \times 5 - 0.2 = 3.8$. Equation (6) gives an incentive margin of -0.4 for the private-proof candidate. Selecting the accepted-verdict route is therefore a profitable unilateral deviation in this declared game, although neither route skips a required check. Appendix B reproduces this calculation using exact fractions.

The protected controller can still enforce the same current authority, exact-order binding and aggregate resource limit for either route. An expired mandate rejects either purchase order. These execution properties do not remove the cost difference between authorised alternatives. This example assigns no separate utility benefit to reduced disclosure. Such a benefit belongs in the principal's payoff or cost model. If confidentiality instead becomes a mandatory constraint that excludes the verdict route, the feasible game changes and its remaining deviations must be analysed again.

All quantities are illustrative utility units and probabilities, not benchmark results. The example motivates measuring the costs and deadlines of the assertion and computation profiles distinguished in Paper 3 [7]. It establishes instability against the displayed deviation; assessing every other permitted program remains necessary for an equilibrium claim.

5 Bounded proofs and private interfaces

5.1 What bounded Löb supplies

The relevant intuition is mutual conditional commitment. One logical program can cooperate if it finds a proof that its counterpart cooperates; the counterpart can use a related rule. Bounded

Löb supplies conditions under which this apparent circularity can be resolved within a specified formal theory and proof-length regime [3]. It is not an instruction to accept an agent's claim that it has proved itself trustworthy.

Here, $\mathcal{T} \vdash \varphi$ means that the statement φ has a derivation in theory $\mathcal{T}$. The symbol $\Box_k \varphi$ expresses a bound on the length of such a derivation. The theory, proof encoding and resource-growth conditions matter. Finding a derivation, checking it and producing a cryptographic argument about the checking computation are separate tasks with separate costs.

Let $\Box_k \varphi$ mean that a proof of φ exists within length k in a specified theory $\mathcal{T}$. Critch's parametric bounded Löb theorem assumes a first-order proof system that represents computable functions, uses logarithmic-length numerals and permits proof abbreviations. Its computable proof-expansion function E satisfies bounded necessitation and bounded inner necessitation. The computable allowance f must asymptotically dominate every function $E \circ h$ with $h \in O(\log k)$, using the conventions of the cited theorem. For a one-variable formula $\varphi(k)$, its schematic implication is [3, Theorem 3]

$$\begin{aligned} \mathcal{T} \vdash \forall k > k_0 (\Box_{f(k)} \varphi(k) \Rightarrow \varphi(k)) \\ \implies \exists k_1 : \mathcal{T} \vdash \forall k > k_1 \varphi(k). \end{aligned} \tag{8}$$

The premise is a theorem inside the formal system, not a declaration that an arbitrary verifier is sound. The result provides neither a general practical proof-search deadline nor a transport theorem for hidden program interfaces. We use it as an established logical baseline. We do not modify its reflection principle or claim that replacing a proof by a succinct argument preserves its premises.

In particular, $\mathcal{T} \vdash \varphi$ is a metatheoretic judgement that φ is derivable in $\mathcal{T}$. The bounded provability formula $\Box_k \varphi$ belongs to the object language. Neither notion is identical to acceptance of a cryptographic argument. The latter establishes a specified computation relation under the argument system's security assumptions. The relation may say that a checker accepted a derivation. It does not make all the checker's possible conclusions true in the intended environment.

5.2 A conditional proof length calculation

Suppose a particular cooperation claim has a supplied proof of length at most αk , with $\alpha \geq 0$. Suppose a verified translation maps this proof to a derivation of the required implementation claim with length at most $a\ell + b$ for an input proof of length ℓ , with $a, b \geq 0$. Suppose the wrapper leaves a proof allowance of $k - d$, with $d \geq 0$.

Lemma 1 Supplied witness budget. If $a\alpha < 1$ and

$$k \geq \frac{b + d}{1 - a\alpha}, \tag{9}$$

the translated supplied proof fits the remaining allowance.

Proof. The translated proof has length at most $a\alpha k + b$. Equation (9) gives $a\alpha k + b \leq k - d$. $\square$

This is a length bound, not a proof-search bound. It also presupposes that the translation establishes the right claim about the implemented program. If $a\alpha \geq 1$, this sufficient calculation may cease to establish eventual fit; it does not prove that no shorter proof exists.

The unresolved bridge is substantive. One must construct the translation, prove its semantic correspondence, account for the change in counterpart information, and establish a feasible

method of obtaining the proof. A public commitment to a secret program provides none of these properties on its own. Equation (9) should therefore be used to reject unsupported resource claims, not to infer a practical value of the bounded Löb threshold.

5.3 Strategically relevant information

Consider two counterpart implementations, P_C and P_D , which respectively cooperate and defect in the active interaction under consideration. Let V_C and V_D denote the complete observations available to a responding agent before it selects its action. They include public statement fields, commitments, certificates, permitted auxiliary information and observable timing.

Theorem 2 Private interface discrimination limit. Suppose the total variation distance between V_C and V_D is at most η . For any response rule A ,

$$\Pr[A(V_C) = C] \leq \Pr[A(V_D) = C] + \eta. \quad (10)$$

Consequently, if the probability of cooperating against P_D is at most δ , the probability of cooperating against P_C is at most $\delta + \eta$.

Proof. Apply the defining total variation bound to the event that the response rule outputs C . Randomised response rules can be included by augmenting both views with the same independent random tape. The second statement follows by substitution. $\square$

What this means. If the responder receives indistinguishable evidence from a cooperative and a defecting implementation, that evidence cannot support substantially different responses to them. Total variation distance bounds the difference in probability of any event inferred from the observations. In the limiting case $\eta = 0$, the responder's cooperation probability is identical in the two cases.

Conditions for use. The premise concerns the complete observation views, including timing and auxiliary information, rather than one matching certificate field. A computational version restricts the responder's computational power and uses the corresponding cryptographic indistinguishability assumption.

For computationally indistinguishable view ensembles, the same argument holds for a probabilistic polynomial-time responder with a negligible distinguishing term. A responder that produced a non-negligible difference would itself be a distinguisher.

The theorem is an application of the standard indistinguishability principle. It is not an impossibility theorem for private cooperation. A valid interface can reveal the relevant distinguishing property without revealing the whole program. Alternatively, enforcement can ensure that every implementation admitted under the same interface satisfies the needed behavioural restriction.

Zero knowledge alone does not imply indistinguishability of all strategically different programs. Public statements may deliberately distinguish them. Conversely, hiding a program while proving an unrelated property cannot supply the information needed to choose a safe cooperative response.

Private booking episode. A certificate that a partner is a valid customer says little about whether its program honours the agreed capacity allocation. Two programs can satisfy that identity statement while making different booking choices. This alone does not prove that their complete views are indistinguishable; it illustrates why the content of the claim matters. A useful interface can disclose or prove the relevant allocation property while concealing private bids or internal inputs. Alternatively, a controller can enforce that property for every admitted booking. Neither approach automatically preserves the original bounded-Löb construction.

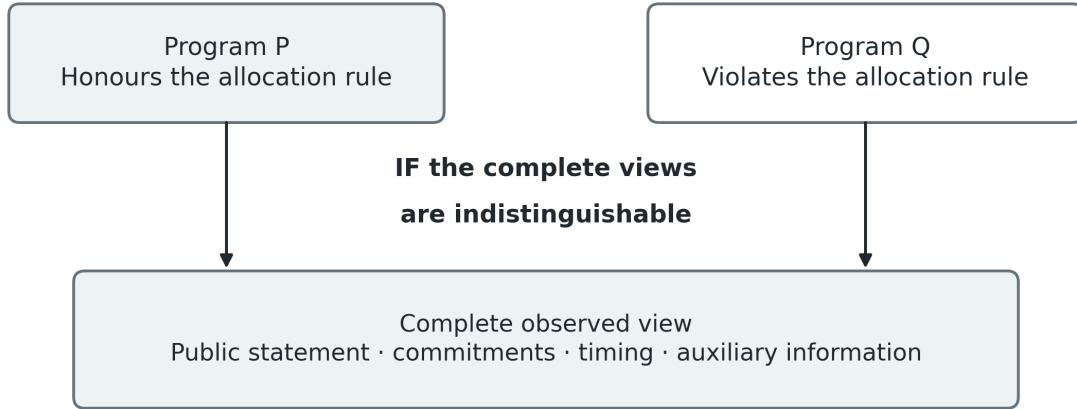
5.4 Historical traces and future action

A finite historical trace has another limit. Let two deterministic programs behave identically for the first m requests. On request $m + 1$, one continues to satisfy a policy and the other violates it. Any certificate whose relation depends only on the common prefix has the same factual basis for both.

This construction does not prevent proving a universal property of a fully specified program.

For example, two deterministic booking programs may respect the agreed allocation for their first ten requests, while one substitutes a different recipient on the eleventh. A proof of those ten past executions does not distinguish their later choices. A proof of an adequate universal program property would supply a stronger basis for future action. It shows why proving the execution of a prefix is a different claim. In deployment, future actions require either an adequate program-level property or continued mediation by a controller whose current guards are checked.

A A conditional limit of private interfaces



A bounded observer cannot reliably discriminate P from Q.

Matching certificate labels alone do not establish this premise.

B Make the relevant property available; check present authority

The property can be public while unrelated private inputs remain hidden.



An epoch-7 proof can remain correct about its historical statement.

After revocation and reissue, execution requires current epoch-8 authority.

Figure 3: The scope of private evidence. A historical or strategically irrelevant statement leaves a different question about the present interaction unanswered. Under Theorem 2’s complete-view premise, a responder cannot reliably distinguish the cooperative and defecting implementations beyond the stated bound. Revealing a relevant property or enforcing it changes the premise. A proof about the current booking still requires current authority, resource state and mediated execution; witness privacy supplies none of those conditions by itself.

6 An epoch bound action controller

6.1 State and trust boundary

A controller is a protected state machine, not an LLM asked to approve another LLM. Its transition rules determine when a request is rejected, reserved, cancelled or committed. Its state includes the current authority period, admitted actors, evidence versions, outstanding reservations and completed action identifiers. The agent generates proposals; the controller holds the capability required to cause the covered effect. Figure 1 shows the separation.

The trusted execution boundary is the collection of components and state whose correctness the guarantee assumes. It need not be a particular hardware enclave. Operating-system isolation, capability enforcement, authoritative storage and the resource adapter must collectively implement the boundary assumed here. Every relevant side-effecting path, including tools or messages that create a covered effect, must be mediated.

We define a transition system for one root mandate. Its state is

$$\Sigma = (e, A, H, v, G, \mathcal{R}, U, s). \quad (11)$$

The root epoch is e . The root activity flag is A . The set H contains admitted instances. The map v records relevant configuration and policy versions. The acyclic grant graph is G . The reservation map is $\mathcal{R}$. The set U contains consumed action identifiers. The cumulative spent quantity is s . Transitions take the authoritative current time as an input; validity checks include grant expiry at that time.

The root mandate has a non-negative budget B . Each reservation r holds a non-negative quantity $b(r)$ and immutable action context. The controller uses exact integer or rational quantities in one declared resource unit. Multiple units require separate accounts and conversion rules. The resource contract assigns each covered action an exact non-negative charge in the declared unit. Its approved payload binds both that charge and the unit; the controller validates them against the authoritative charging rule. For each reservation, $b(r)$ equals this charge, and the atomic resource adapter performs precisely the associated effect and charge. This correspondence connects the ledger inequality to the covered resource. Metered or uncertain consumption requires a separate reservation and settlement model.

A certificate is an abstract checked record binding an instance, grant path, action identifier, exact payload, audience, configuration version, policy version and epoch. Grant records and their scope sets are immutable; changing a grant requires a new record and a newly validated path. Certificate soundness is idealised in this section. Cryptographic failure is treated separately in Section 7.

The trusted boundary contains the controller, its current state and the resource operation that implements the covered digital effect. Every covered effect is mediated. A process able to write directly to the resource lies outside this assumption. So does a sensor whose false report changes the meaning of a checked predicate.

The canonical initial state has zero spending, no reservations and no consumed identifiers. Admitted instances and grant records, if initially present, satisfy the stated authority and scope conditions. More generally, a valid starting state must satisfy the resource bound and authority consistency properties and have a valid prior history for any existing records; the displayed proof uses the canonical empty ledger.

6.2 Transitions

Admit and delegate. A current authorised transition creates a fresh instance or child grant. A child grant’s action scope is a subset of its validated parent’s scope and the local policy. Its expiry is no later than its parent’s. Scope inclusion inherits per-action amount restrictions. Separate cumulative limits for descendants require additional shared accounts and invariants. Admission adds no funds to the root budget. Distinct descendants share the same root reservation account.

Certify. The decision procedure checks the intended action against a specified policy and snapshot. It returns a record with exact context bindings. Producing that record creates no resource effect and does not reserve a budget.

Reserve. The controller checks root activity, the current grant path, instance status, certificate context, unused action identifier and the absence of a duplicate outstanding reservation. It then atomically checks and updates

$$s + \sum_{r \in \mathcal{R}} b(r) + b_{new} \leq B. \quad (12)$$

The stored reservation binds the checked action and context. The check and the update form one atomic operation.

Commit. The controller rechecks current authority and relevant configuration, confirms exact equality with the approved action, and checks that the identifier is unused. In one atomic transition, it consumes the reservation, adds its quantity to s , records the identifier in U , performs the covered digital effect and records its outcome.

The commit-time authority check explicitly includes root activity, current instance status, every grant on the accepted path and each grant’s expiry at the authoritative current time. Configuration and policy versions must both match the reserved context. A relevant policy revision advances its bound version, so dependent reservations require renewed evidence before commit. The final checks and the covered effect share the same atomic transition.

Cancel. A current reservation can be released without creating an effect. A completed action cannot be cancelled in this manner. If an external resource has an unresolved outcome, its reservation remains held until the outcome is resolved through a separately specified protocol.

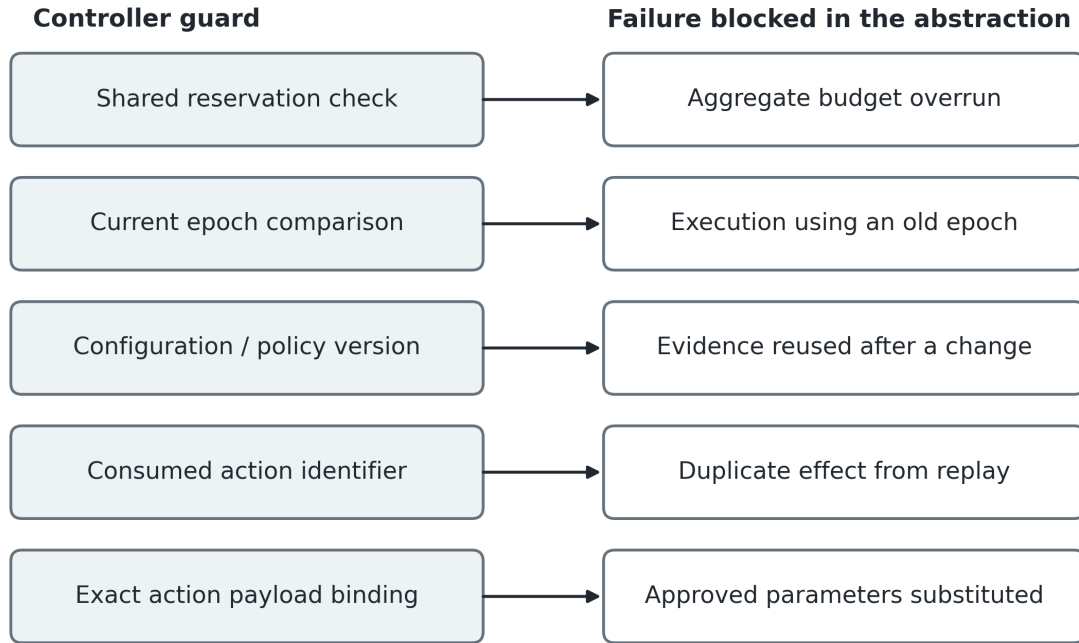
Revoke. An authenticated authority deactivates the root, advances its epoch and cancels its uncommitted reservations. Reissue requires a separate authorised transition. Reissue uses the new epoch and retains the budget and spending ledger; old certificates do not regain validity. The authenticated reissue policy determines which existing grant paths remain eligible for fresh certification. Every path must satisfy that policy, current expiry and scope checks before use; setting the root activity flag alone does not authorise every previous delegation.

Update or retire. A relevant configuration update changes the instance’s version. Retirement removes the instance from the admitted set and releases its uncommitted reservations. A changed instance requires renewed evidence before another dependent commit. Neither transition resets cumulative spending.

Shared-budget episode. One organisation gives two worker instances a common budget of 100 booking credits. Both propose a 60-credit booking. The first reservation changes outstanding reservations from 0 to 60, leaving 40 available. A second 60 reservation fails because $0 + 60 + 60 > 100$. On committing the first booking, spent becomes 60 and reserved becomes 0; total committed and reserved exposure remains 60. The check and update must be atomic even when the requests arrive concurrently. A fresh child instance inherits access to the shared account, not a fresh 100 budget.

Epoch episode. A certificate is produced in authority epoch 7. Revocation deactivates that authority, and reissue uses epoch 8. The old proof can remain correct about its original statement while failing the present epoch check. Epochs identify authority generations; they are distinct from configuration versions and wall-clock expiry.

A Five highlighted guards in the specified controller



B A shared mandate of 100 credits; spent = 0

After request 1 reserves 60:

Reserved: 60	Available: 40
---------------------	----------------------

Request 2 asks for 60: $0 + 60 + 60 = 120 > 100$ **Reject**

The reservation check and update must be atomic.

Figure 4: Five highlighted controller guards. Aggregate reservation prevents the represented budget overrun; current-epoch, version, used-identifier and payload checks address the corresponding stale-authority, stale-configuration, replay and substitution failures. These are the five controls removed individually in Section 8.4. Their effect is conditional on the full model: valid authority roots and grant paths, scope checks, admitted instances, expiry checks, sound certificate checking, complete mediation and atomic transitions. The omissions establish counterexamples in this design, not universal necessity or sufficiency of five checks.

This model has fixed principals but may admit and retire bounded numbers of instances acting for them. Introducing a new independent principal changes the strategic game and requires a new equilibrium analysis.

6.3 Invariants

Theorem 3 Controller safety. Assume a valid initial state, sound certificate checking, correct grant-root information, complete mediation, and atomic transitions as defined above. Every reachable state and executed action satisfy the following properties:

1. The aggregate resource invariant $s + \sum_{r \in \mathcal{R}} b(r) \leq B$ holds.
2. Each covered effect has current authority and exactly the parameters bound to its reservation at the commit point.
3. An action identifier produces at most one covered effect.
4. A delegated scope remains within its validated ancestors' scopes.
5. Once revocation takes effect in the authoritative state, no later commit can use the old epoch, including after reissue.

Proof. Initially there is no spending or reservation, so the resource invariant holds. Admission, certification and configuration changes do not increase the resource total. A reservation is added only if Equation (12) holds. Commit removes $b(r)$ from reservations and adds the same quantity to spending, leaving the total unchanged. Cancellation, retirement and revocation remove reservations without reducing recorded spending. The resource invariant therefore holds by induction.

Only Commit creates a covered effect. Its guards establish current activity, the valid grant path and expiry, a live instance, matching configuration and policy versions and epoch, and equality of the executed action and resource charge with the reserved record. These facts prove the second property at the atomic commit point.

Commit requires that the identifier be absent from U and adds it to U atomically. No transition removes an identifier from U . Therefore a later commit with the same identifier fails, proving the third property.

Every new delegation is checked for scope inclusion. Existing grant records and their scopes do not change through any other transition. Transitivity of set inclusion gives the fourth property for the acyclic ancestor chain. Replacement grants require a newly validated path before use.

Revocation advances the authoritative epoch. Commit requires equality between the record's epoch and the current epoch. Epochs are never reused, so an old record fails after revocation and also after reissue. This proves the fifth property. $\square$

The proof uses standard invariant reasoning. Its operational content lies in the stated guards and atomicity boundary. It does not infer a real-world implementation's linearizability from the existence of this transition specification.

What this means. The proof constrains every covered effect, regardless of the proposer's suggested rationale. Certification establishes an accepted record. Reservation accounts for a pending commitment. Commit rechecks current conditions and creates the effect. These stages have different meanings.

Conditions for use. The theorem assumes the complete transition specification and idealised certificate soundness. It covers the stated resource and authority properties, not every possible harm or every external resource implementation. A local ledger update followed by an ordinary remote API call is not automatically one atomic covered effect. Replay protection applies to an action identifier. Preventing equivalent business actions under different fresh identifiers requires a separate deduplication rule.

Corollary 3 Proposal sequence independence. Under the assumptions of Theorem 3, its invariants hold along every execution generated by arbitrary proposal sequences and permitted

administrative transitions, including sequences selected adaptively by probabilistic or adversarial proposers. Rejected requests create no covered effect and are modelled as state-preserving steps.

Proof. Fix a realised sequence of requests and an ordering of permitted transitions. Starting from a valid state, each accepted transition preserves the invariants by Theorem 3. Each rejected request preserves the state. Induction establishes the invariants at every finite prefix, independently of how the requests were chosen. Any distribution over these permitted executions therefore assigns probability one to the invariant event in the ideal model. $\square$

This is a consequence of the existing invariant argument. It does not extend the deterministic equilibrium result to arbitrary probabilistic games. Cryptographic failures, compromised controllers and bypass paths are outside its ideal premises and require separate treatment.

6.4 Trace refinement and distributed revocation

Project each controller execution onto its Commit events, retaining their action and authority context. Every projected effect has the authority and exact binding established by Theorem 3. Action identifiers are not reused; delegated scope and epoch restrictions hold at each commit point; and cumulative committed charges remain at most B . The stronger bound on spent plus reserved quantities holds along the full controller execution. Administration and reservation steps create no covered effect, while their authoritative state changes determine which later Commit events are permitted. This establishes safety trace inclusion for the induced abstract effect specification.

A state-based refinement of a separate implementation additionally requires its transition relation, an abstract-state mapping and a proof that the mapping preserves the required observations. Relating it to the implemented strategic game additionally requires matching its resolution procedure, abort outcomes, timing and costs. The transfer theorem below states those extra obligations.

In a distributed deployment, revocation has a delay. If every commit point adopts the new authoritative epoch or stops committing within a declared bound Δ , old-epoch commits cease by that bound. This is a conditional statement about the propagation and fencing protocol. It does not undo effects already committed.

A network partition can prevent current-state confirmation. Continued availability must then be limited by a stated expiry or other justified rule. The abstract controller provides no guarantee of instantaneous global revocation. Likewise, a remote side effect outside its atomic resource boundary requires a separate execution and outcome protocol.

7 Cryptographic realisation and equilibrium transfer

7.1 The certified relation

A possible public statement binds a program or controller commitment, configuration commitment, policy version, principal and instance context, audience, action digest including its resource charge and unit, root epoch and identifier. A witness contains the required credentials and checked derivation or policy execution trace.

An argument system establishes membership in this specified relation. A zkVM is one possible implementation of its checker [24].

A zero-knowledge virtual machine supports a cryptographic argument about a specified program execution. The prover supplies a witness, such as private credentials and a policy trace. The verifier checks the argument against a public statement that binds the relevant program, action and context. Zero knowledge, when provided by the chosen construction, limits disclosure of the witness; soundness concerns acceptance of false statements under the security assumptions. These are separate properties.

The narrow kernel evaluates the declared policy relation. The prover produces evidence about that computation. The verifier validates the evidence. The state controller then decides whether the exact action may proceed against current authority and resource state. Direct trusted evaluation of the kernel is another design option; a zkVM is useful when independent verification or witness privacy justifies its costs.

Cryptographic randomness in proof generation is compatible with a deterministic policy kernel. It need not change the policy decision being proved. Conversely, proving one model rollout with one random seed does not prove that seeds were sampled fairly or that other rollouts satisfy the same property. Any randomness claim must be included in the defined relation and supported by its input-generation protocol. A formally specified policy language can reduce ambiguity in the relation being checked [25]. Neither choice proves that a supplied fact is true or that an external effect occurred.

The cryptographic requirements include commitment binding, signature authenticity, argument soundness and context binding. Witness privacy is a separate property. The protocol also needs replay state and current authority state. A public hash of a low-entropy confidential value may reveal it through guessing; hiding commitments require a suitable construction.

This paper does not instantiate or benchmark a complete cryptographic protocol. The theorem below is a reduction target for such an implementation. Its premises are stronger than possession of an accepted proof.

7.2 Cost aware simulation

Let V_i be the ideal revocation-aware game utility. Let $\tilde{V}_i$ be concrete expected utility and let Π^* implement the ideal candidate p^* . Suppose

$$\tilde{V}_i(\Pi^*) \geq V_i(p^*) - \rho_i^*. \quad (13)$$

For every admissible concrete deviation A_i , suppose there is a permitted ideal deviation d_i such that

$$\tilde{V}_i(A_i, \Pi_{-i}^*) \leq V_i(d_i, p_{-i}^*) + \rho_i(A_i). \quad (14)$$

The ideal deviation must account for the concrete adversary's permitted aborts, messages, scheduling effects and costs. A simulator that reproduces only successful outputs does not establish Equation (14).

One sufficient source of these errors is a coupling of the entire payoff-relevant outcomes, including cancellation and timing effects. Evaluate both coupled outcomes with the same gross-payoff function on a common outcome space and utility scale. If outcomes disagree with probability at most η_i , this function takes values in a common interval of width at most W_i , and the absolute difference of expected costs is bounded by κ_i , then a corresponding error bound is

$$\rho_i \leq W_i \eta_i + \kappa_i. \quad (15)$$

The range must include bad-event outcomes. Rare events with unbounded losses cannot be hidden inside a finite value of W_i . Statistical coupling is stronger than computational indistinguishability. In the computational version, a comparable expected-payoff bound requires bounded, efficiently computable utility and the appropriate simulation-based security statement. To infer a negligible utility error from a negligible distinguishing advantage, the utility range must also be polynomially bounded in the security parameter; otherwise its growth must remain explicit in the error bound.

7.3 The transfer result

For this transfer statement, define $M_i(d_i) = V_i(p^*) - V_i(d_i, p_{-i}^*)$ using the actual ideal utility model. Equation (6) gives this difference in the constant-abort baseline; Equation (7) gives it when abort payoffs depend on the profile.

Theorem 4 Conditional equilibrium transfer. Under Equations (13) and (14), the gain from a concrete deviation is at most

$$-M_i(d_i) + \rho_i^* + \rho_i(A_i). \quad (16)$$

If the candidate is an ε -equilibrium of the ideal revocation-aware game and both errors are bounded by ρ , its concrete implementation is an $(\varepsilon + 2\rho)$ -equilibrium against the declared deviations.

Proof. Subtract Equation (13) from Equation (14). The ideal difference is $V_i(p^d) - V_i(p^*) = -M_i(d_i)$, giving Equation (16). An ideal ε -equilibrium has $-M_i(d_i) \leq \varepsilon$ for every permitted deviation. Bounding each implementation error by ρ proves the result. $\square$

What this means. If implementation changes the candidate's utility and the deviator's utility by bounded amounts, their combined effect limits how far an ideal incentive guarantee can weaken. For illustration, an ideal margin of 0.2 and two valid error bounds of 0.03 leave a concrete deviation gain at most -0.14 . These numbers illustrate the inequality; they are not measured security errors.

Conditions for use. Every concrete deviation needs an admissible ideal counterpart with the required one-sided utility bound. Timing, cancellation, costs and bad-event outcomes belong in that correspondence. An accepted cryptographic proof or a matching successful output alone does not establish the premise.

The general simulation and perturbation principle is established territory, including cost-aware computational implementation [9]. The present application inserts the explicit revocation and cost margin from Equation (6). It identifies how much implementation error a particular commitment can tolerate.

For instance, a strictly positive margin can absorb a bounded implementation error. A zero-margin profile generally transfers only to an approximate equilibrium unless the relevant errors vanish or a sharper one-sided argument is available. Cryptographic security alone does not remove a non-negligible timing advantage or cost saving.

The polynomial-time restriction is independent of bounded Löb. A finite search over all proofs of length k need not run in time polynomial in λ . A computational-security claim therefore requires an explicit resource relation, rather than silently identifying these parameters.

7.4 What a full realisation still requires

Theorem 3 provides an ideal action controller. Theorem 4 additionally requires a strategy correspondence and quantified errors. A bounded-storage implementation must also specify its mandate horizon, identifier namespace, replay-state retention and behaviour on capacity or epoch-counter exhaustion. The ideal model never removes used identifiers or reuses epochs. Any compaction or reset therefore needs its own preservation argument. A concrete implementation must show that every allowed adversarial trace maps to the specified ideal game, or identify the additional outcome it creates. It must preserve the relevant cost accounting and authority timing.

Complete mediation, current-state checks and conserved budgets can rule out classes of unauthorised effects. They do not prove that a cooperative strategy is optimal, that every issuer’s statement is accurate, or that the policy protects all affected parties. Those claims require their own premises.

8 Reproducible finite analysis

8.1 Exact utility comparisons

The executable listing in Appendix B checks Equations (6) and (7) using Python’s rational arithmetic. The grid takes candidate and deviating active payoffs from $\{0, 1, 2, 3, 4, 5\}$; the candidate abort payoff from $\{0, 1, 2\}$; both completion probabilities from $\{0, 1/5, 2/5, 3/5, 4/5, 1\}$; candidate cost from $\{0, 1/10, 2/10, 3/10, 4/10\}$; and deviating cost from $\{0, 1/10\}$. This gives 38,880 parameter combinations.

For each combination, the script checks that direct subtraction equals Equation (6) and that its sign gives the same comparison of utilities. It also checks Equation (7) with the deviating abort payoff set to two minus the candidate abort payoff. All assertions passed. These are finite checks of algebraic consistency, not statistical estimates of the frequency of incentive failures.

The script also reports the four unilateral comparisons in Table 5 and the additional evidence-route comparison in Section 4.6. The common-survival and faster-deviation cases use the same active payoffs and costs; only the deviating completion probability changes.

Table 5: Exact unilateral comparisons with zero abort payoffs

Case	Candidate utility	Deviating utility	Margin
Optional checking	29/10	3	-1/10
Common survival	4/5	3/10	1/2
Faster deviation	4/5	1	-1/5
Positive margin	23/10	9/10	7/5

A positive entry establishes stability against the displayed deviation only. It does not by itself establish equilibrium against the entire program class.

8.2 A finite response table game

To check a complete but small deviation set, consider three program labels R, C, D . Program R cooperates against R and C , and defects against D . Program C always cooperates; program D always defects. The active payoffs are the Prisoner’s Dilemma values: mutual cooperation gives 3, unilateral defection gives 5 to the defector and 0 to the cooperator, and mutual defection gives 1.

These are explicit response-table programs. In particular, R is not a simulation of FairBot, a theorem prover or an LLM. The table deliberately supplies behaviour directly so that the effect of timing and costs can be isolated.

The script enumerates all nine ordered profiles and tests every pure unilateral deviation by each player. Table 6 gives all pure equilibria in each regime. Mixed equilibria are not enumerated.

Table 6: Pure equilibria in the three program game

Regime	Completion probabilities	Costs	Pure equilibria
Cost free	$q = 3/10$ throughout	All zero	(R, R) and (D, D)
Optional cost	$q = 3/10$ throughout	R costs $1/10$; others zero	(D, D)
Uniform cost	$q = 3/10$ throughout	Every program costs $1/10$	(R, R) and (D, D)
Unequal completion	$q(R, R) = 3/10$; otherwise $q = 1$	Every program costs $1/10$	(D, D)

Abort payoffs are zero. The second regime removes (R, R) because switching to C preserves cooperation while avoiding cost. The fourth removes it because switching to C preserves the active payoff and increases the completion probability. The uniform-cost regime is a controlled calculation, not a claim that real proof-search costs can be equalised without additional assumptions.

8.3 Exhaustive controller instances

The finite controller abstraction has two possible instances, one immutable action identifier per instance, unit-sized actions, one possible configuration update per instance, and one root revocation followed by one possible reissue. Each instance can be admitted once and retired once. The root budget is either one or two units.

The explored state records root phase, instance lifecycle, configuration versions, available certificate contexts, reservations, used identifiers and cumulative spending. Operations are admission, retirement, configuration update, certification, reservation, cancellation, execution, revocation and reissue. The baseline permits execution only of the payload bound to the certificate. A breadth-first search explores each reachable state once and checks all generated transitions.

Table 7: Complete baseline explorations

Root budget	Reachable states	Examined transitions	Violations
1	1,427	3,714	0
2	1,819	4,962	0

This abstraction checks aggregate accounting, replay, current epochs, configuration freshness and payload binding. It does not represent a general delegation graph, continuous expiry times, signature verification or a distributed resource protocol. Consequently, Table 7 is not an exhaustive exploration of every system covered by the informal transition specification. The general invariant argument is the separate induction in Theorem 3.

8.4 Deliberate control omissions

Five modified versions each remove one control. Exploration stops on the first violation and returns a shortest counterexample in number of transitions from the initial state. The replay case uses budget two; the other cases use budget one.

Table 8: Counterexamples produced by control omissions

Omitted control	First violated property	Trace length
Aggregate reservation guard	Root budget exceeded	6
Epoch comparison	Old certificate executes after reissue	6
Configuration comparison	Old configuration certificate executes	5
Used identifier guard	Same identifier executes twice	6
Exact payload binding	Approved payload is substituted	4

For the epoch omission, the trace admits an instance, certifies its action in epoch zero, revokes the root, reissues at epoch one, reserves using the old certificate, and executes. A signature or proof about the old record can remain valid while the action is no longer authorised by that record.

For the budget omission, two admitted instances each obtain a certificate and reserve one unit against a one-unit root. No individual action exceeds the root’s nominal limit; the combined reservations do. The failure therefore appears before execution. A separate per-instance ceiling cannot replace shared accounting for a shared mandate.

Removing a guard is not intended to approximate an observed implementation defect rate. It demonstrates that the selected guard matters in this model and that the checker can expose the corresponding failure. No claims about real-world attack success rates follow from these runs.

8.5 Reproduction

Appendix B contains the complete executed listing. Save it as `paper1_checks.py` and run it with Python 3.10 or later using normal assertion checking. It uses only the standard library, no network access and no random seed. It prints the results and writes `paper1_check_results.json` beside the script. The tables above can be recovered from that output. The code is included to make the finite analysis auditable; it is not a deployed authorisation service.

9 Implications for agent assurance

9.1 Cooperation is not a safety specification

A cooperative equilibrium can be harmful to parties outside the game. A Nash equilibrium only excludes profitable unilateral deviations by represented players. It does not establish a socially desirable outcome, resistance to coalitions, or the validity of the players' objectives.

The controller's constraints must therefore be specified independently of the claim that a profile is cooperative. An external safety rule may exclude an action even when every participating principal prefers it. Conversely, the controller can permit several actions without establishing that the selected action is optimal or aligned. These distinctions prevent a game-theoretic certificate from being interpreted as a general safety certificate.

An authorised robot command illustrates the boundary. A controller may correctly check the operator, command parameters and current authority while a collision remains possible because relevant physical conditions are absent from its model. Such a deployment needs its own hazard specification, adequate observations and physical safety controls. The digital booking theorem does not establish those conditions. Similarly, organisations can cooperate on an outcome that harms non-participants; equilibrium must not stand in for an outsider-protection policy.

9.2 The immediate engineering boundary

The immediate implementation target is a narrow deterministic decision kernel connected to a protected state controller. In the running example, the kernel checks a booking's declared policy relation and evidence, while the controller checks current authority, reservations and commit conditions. A generative model proposes the booking and may supply candidate evidence; it holds no bypass capability for the covered booking operation. This is an application scenario, not an experiment reported in this paper.

Identity identifies the represented principal and instance. Authorisation determines the permitted action in the current context. Provenance links an accepted record to the program, policy and inputs that produced it. None of these alone establishes future behavioural compliance. A certificate of a past decision must be joined to current-state checking and complete mediation of the next covered effect.

The cross-domain architecture agenda in [26] motivates portable authority and evidence. Portability must not erase root accounting or turn a fresh authority query into a permanently reusable entitlement. A receiving controller needs an agreed interpretation of action scope, versioning, issuer authority and revocation. Semantic interoperability is an additional specification problem, not a property obtained from signatures alone.

The analysis does not require proving the entire inference computation of a large model. The certified relation can instead concern a narrow decision kernel and its declared evidence. This reduces the statement being proved, but also narrows what the proof establishes. It cannot silently inherit claims about the model's motives or unmediated behaviour.

9.3 Resulting constraints for probabilistic AI agents

The proposed architecture permits probabilistic action generation while requiring every covered effect to pass through current-state enforcement. Corollary 3 makes the scope precise: under the ideal controller assumptions, the resource and authority invariants hold for every admitted

execution. The proof does not require a distribution over the agent’s proposals or an interpretation of its motives. Denial, timeout or failed certification produces no covered effect.

Strategic claims require more information. A stochastic extension must specify joint laws for unrevoked completion time T_p , gross completion payoff $Y_i(p)$, revocation time R and realised implementation cost $C_i(p)$. With constant abort payoff a_i , an integrable all-or-abort model has expected utility

$$V_i(p) = a_i + \mathbb{E} \left[\mathbf{1}_{\{T_p < R\}} (Y_i(p) - a_i) - C_i(p) \right].$$

Here the indicator is one when completion precedes revocation and zero otherwise. Correlation between completion, outcome and revocation remains inside the expectation. If the completion payoff is fixed at $u_i(p)$ and completion time is the deterministic $t(p)$, the expression reduces to Equation (3). Substituting an average time into the survival function generally gives a different quantity from averaging survival across random times.

For example, under the running revocation law, let a profile finish at time 0.5 or 2 with equal probability, independently of revocation. Its completion probability is $0.5(1) + 0.5(0.3) = 0.65$. Its mean completion time is 1.25, whose survival probability is only 0.3. This calculated example shows why a mean-time substitution loses relevant information.

The displayed expression defines an extension of the payoff model, not an additional general equilibrium theorem. Stochastic counterpart responses, shared randomness and adaptive choices require their own game specification. Bounded-Löb and cryptographic-security premises must also be retained rather than inferred from a fixed-seed execution.

9.4 Applicability to multi agent systems

Multiple processes need not create multiple independent players. The distinction is between workers acting under an existing principal and new principals with their own preferences and authority. Table 9 identifies what the present model supports and what requires an extension.

Table 9: Applicability to changing agent systems

System feature	Requirement or limit
Several workers for one principal	Maintain common authority ancestry, spending and reservations
Bounded admission and retirement	Apply governed lifecycle transitions; creating a child adds no funds
Probabilistic proposals	Preserve all controller guards for every admitted request; model distributions separately for incentives
A new independent principal joins	Redefine the strategic game and reassess equilibrium
Several controllers use one shared mandate	Supply a justified state-consistency protocol or allocate disjoint portions of the resource
Configuration, memory or tool changes	Determine which evidence claims depend on the change and renew or constrain them
Coordinated harmful actions	Specify the relevant joint safety properties; individual permission and shared-budget control cover only part of the problem

Cross-organisation cooperation also requires a common interpretation of action scope, resource units and cancellation rights. DIDs, credentials and evidence provenance can carry attributable claims, but the receiving system must still apply its own current policy and validate the authority chain. The present game assumes the relevant semantics; it does not derive them from signatures.

9.5 Monitoring evaluation and behavioural change

Per-instance evaluation does not establish a collective resource invariant. Recent multi-agent control experiments show that distributing an attack across instances can weaken per-instance monitoring in the studied environments [27]. The controller’s shared budget addresses one explicit joint property. It does not solve arbitrary collusion, distributed deception or unmodelled channels.

Testing, evaluation, verification and validation have different roles here. Evaluation can measure program behaviour in selected environments. Verification can establish a controller invariant under a formal transition model. Validation must examine whether that model, its authority roots and its effect boundary represent the intended deployment. A passing finite test is neither a universal proof nor evidence that the policy is adequate.

A configuration change can invalidate a certificate even without statistically detectable behavioural drift. Conversely, behaviour can change through inputs, memory or external state while a configuration identifier stays constant. Relevant state must therefore either be bound into the certified relation or constrained at the action boundary. The present paper does not estimate drift rates or monitoring error.

Paper 2 develops the runtime-assurance continuation for these omissions, using a joint information model, pending-effect retention and a permanently charged episode account [6]. Paper 3 develops typed evidence validation and private verification across organisations [7]. A resource reservation here may be released before its atomic effect; an issued assurance debit in Paper 2 remains charged. Instance admission here also differs from operation admission in Paper 3, whose remote effect may occur later. Each transfer therefore retains its stated state, timing and probability assumptions. Empirical model adequacy, mathematical correctness and cryptographic soundness remain distinct obligations.

9.6 Execution safety and useful progress

A safety invariant prevents a specified forbidden state or effect. A progress claim requires that something useful eventually happens. A controller that indefinitely rejects every booking can preserve the budget while failing the service’s purpose. Likewise, an agent can hold reservations or submit repeated rejected requests without causing the particular overspending violation. Reservation expiry, fair scheduling, request limits and recovery rules therefore need their own specifications and incentive analysis.

The distinction is established in the execution-monitoring literature [19]. Here the controller theorem covers the stated invariant properties. It does not establish availability, freedom from denial of service, eventual agreement or economically useful operation. Stronger controls may also change timing and costs, so their strategic effects return to the margin in Equation (6).

10 Limitations and further research

The equilibrium criterion is algebraically elementary. The indistinguishability argument and equilibrium perturbation bound instantiate standard principles. Their intended contribution is a precise analysis of what changes when program commitments are deployed under revocable authority, not a replacement for the foundational results cited above.

The program game has a fixed finite set of principals and a declared bounded deviation class. Dynamic admission concerns instances acting for those principals. Open entry by new principals, coalitional deviations, repeated reputation effects and strategic manipulation of the revocation mechanism require different games.

The bounded Löb discussion is conditional. No proof-preserving compiler from a concrete Löbian agent to a private cryptographic interface is constructed. No numerical threshold for feasible proof search is claimed. Establishing such a compiler, or a restricted impossibility result, would be a separate substantive contribution.

The controller assumes an authoritative state and atomic covered effects. It does not prove a distributed implementation linearizable, guarantee timely global revocation under arbitrary partitions, or resolve uncertainty about an external physical action. It also does not defend a compromised root authority. These are boundaries of the claimed guarantee, not implementation details that can be omitted from a deployment argument.

The proofs are written mathematical arguments, not proof-assistant-checked derivations. The finite searches validate two small executable abstractions and five mutations. They do not replace the induction proof, establish the absence of implementation bugs, or cover every instance count and delegation structure.

No full cryptographic protocol is instantiated. The simulation errors in Theorem 4 are assumptions to discharge, not measured security parameters. An implementation study would need to quantify proof-generation and verification costs, certificate and policy-update latency, revocation delay, abort handling and the full set of adversarial deviations. Comparing only successful proof verification times would miss several terms in the incentive margin.

The remaining end-to-end research step across the series is deliberately small: one finite program game, one specified assurance continuation, a bounded evidence checker, a mediated digital resource and a cost-aware implementation relation. Papers 2 and 3 develop the continuation and evidence interfaces [6, 7]. Their deployment must still discharge controller refinement, stochastic-transfer and strategy-correspondence obligations, then measure the costs that enter Equation (6). Expansion to richer agent systems requires those assumptions to remain explicit under composition.

A Research sequence and claim boundaries

Step 1 Define the question. Separate cooperation in a program-selection game from authority to execute a particular action. Restrict this paper to fixed principals, bounded programs and mediated atomic digital effects. Companion Papers 2 and 3 develop runtime assurance under partial observation and private cross-domain evidence validation [6, 7]. Their additional premises remain outside the claims proved here.

Step 2 Establish prior art. Examine the logical and simulation-based program-equilibrium literature, then computation costs, conditional disclosure and secure implementation. Compare current proof-carrying agent proposals with earlier proof-carrying data. Retain the distinctions between published theorems, preprints and infrastructure specifications.

Step 3 Test the central intuition. Make verification costs and authority windows explicit. Construct a cheaper outcome-equivalent deviation and a faster deviation under the same external revocation law. Derive the exact margin before making a preservation claim.

Step 4 State the implementation obligations. Specify current-epoch checks, exact action binding, shared reservations, replay state and the atomic effect boundary. Prove the invariants by induction. State equilibrium transfer conditionally on a strategy correspondence that preserves relevant timing and costs.

Step 5 Execute finite checks. Check 38,880 rational parameter combinations, enumerate the pure equilibria of four three-program games, and exhaust two finite controller instances. Remove five controls individually and retain the counterexample traces. Publish the complete listing in Appendix B.

Step 6 Review the claims. Do not identify a historical trace with a universal behavioural guarantee, an accepted cryptographic argument with object-level provability, or a safety invariant with equilibrium. Keep the proof-translation and concrete realisation obligations open.

Table A1: Assumptions required by each claim

Claim	Critical premise	Failure outside the premise
Exact margin	Payoff and cost model in Equation (3)	Missing partial outcomes or correlated effects
Proof length fit	Correct supplied translation and stated expansion	No guaranteed witness or search deadline
Private interface limit	Complete views within the stated distance	Revealing statements can distinguish programs
Controller invariants	Complete mediation and atomic current-state guards	Bypass, stale state or external unresolved effects
Equilibrium transfer	Every concrete deviation has a cost-aware ideal counterpart	New profitable scheduling or abort behaviour

B Executable finite checks

The following listing is the complete script used for the reported finite results. It should be copied as plain text with its indentation preserved. It is independent of external packages. Python assertions must remain enabled.

```
"""Exact finite checks for Paper 1. Python standard library only.

These checks exercise the specified finite abstraction. They are not a
proof-assistant formalisation, a cryptographic implementation, or AI trials.
"""

from collections import deque
from dataclasses import dataclass, replace
from fractions import Fraction as F
from itertools import product
import json
from pathlib import Path

def incentive_checks():
    count = 0
    for us, ud, abort, qs, qd, cs, cd in product(
        range(6), range(6), range(3),
        [F(i, 5) for i in range(6)],
        [F(i, 5) for i in range(6)],
        [F(i, 10) for i in range(5)], [F(0), F(1, 10)]
    ):
        vs = abort + qs * (us - abort) - cs
        vd = abort + qd * (ud - abort) - cd
        margin = qs * (us - ud) - (qd - qs) * (ud - abort) - (cs - cd)
        assert vs - vd == margin
        assert (vs >= vd) == (margin >= 0)
        other_abort = 2 - abort
        vd_other = other_abort + qd * (ud - other_abort) - cd
        extended = (qs * (us - ud) - (qd - qs) * (ud - other_abort)
                    + (1 - qs) * (abort - other_abort) - (cs - cd))
        assert vs - vd_other == extended
        count += 1
    return count

def finite_games():
    names = ("R", "C", "D")
    responses = {
        "R": {"R": "C", "C": "C", "D": "D"},
        "C": dict.fromkeys(names, "C"),
        "D": dict.fromkeys(names, "D"),
    }
    gross = {("C", "C"): 3, ("C", "D"): 0,
             ("D", "C"): 5, ("D", "D"): 1}
    cases = []
    for regime in ("cost free", "optional cost", "uniform cost",
                  "unequal completion"):
        def payoff(p, opponent):
            q = F(3, 10)
            if regime == "unequal completion" and (p, opponent) != ("R", "R"):
```

```

        q = F(1)
        cost = F(0)
        if regime == "optional cost" and p == "R":
            cost = F(1, 10)
        if regime in ("uniform cost", "unequal completion"):
            cost = F(1, 10)
        return q * gross[responses[p][opponent],
                        responses[opponent][p]] - cost
    equilibria = []
    for p, q in product(names, repeat=2):
        if (all(payload(p, q) >= payload(d, q) for d in names)
            and all(payload(q, p) >= payload(d, p) for d in names)):
            equilibria.append([p, q])
    cases.append({"regime": regime, "pure_equilibria": equilibria})
return cases

@dataclass(frozen=True)
class State:
    phase: int = 0 # 0 initial authority; 1 revoked; 2 reissued at epoch 1
    life: tuple = (0, 0) # 0 absent; 1 admitted; 2 retired
    version: tuple = (0, 0)
    cert: tuple = (-1, -1) # encoding 2*epoch + configuration version
    held: tuple = (-1, -1) # reserved immutable certificate context
    used: tuple = (0, 0) # one immutable action identifier per actor
    spent: int = 0

def put(values, i, value):
    v = list(values)
    v[i] = value
    return tuple(v)

def current(s, c, i, mutant):
    if c < 0 or s.phase == 1 or s.life[i] != 1:
        return False
    epoch = int(s.phase > 0)
    return ((mutant == "epoch" or c // 2 == epoch)
            and (mutant == "configuration" or c % 2 == s.version[i]))

def transitions(s, budget, mutant):
    """Yield operation, next state, and any forbidden executed effect."""
    if s.phase == 0:
        yield "revoke root", replace(s, phase=1, held=(-1, -1)), None
    if s.phase == 1:
        yield "reissue root at epoch 1", replace(s, phase=2), None
    for i in range(2):
        if s.life[i] == 0 and s.phase != 1:
            yield f"admit {i}", replace(s, life=put(s.life, i, 1)), None
        if s.life[i] == 1:
            yield f"retire {i}", replace(
                s, life=put(s.life, i, 2), held=put(s.held, i, -1)), None
        if s.version[i] == 0:
            yield f"update configuration {i}", replace(
                s, version=put(s.version, i, 1)), None
        if s.life[i] == 1 and s.phase != 1 and not s.used[i]:

```

```

        c = 2 * int(s.phase > 0) + s.version[i]
        if s.cert[i] != c:
            yield f"certify {i} context {c}", replace(
                s, cert=put(s.cert, i, c)), None
        if (s.held[i] < 0 and current(s, s.cert[i], i, mutant)
            and (not s.used[i] or mutant == "replay")):
            total = s.spent + sum(c >= 0 for c in s.held)
            if total + 1 <= budget or mutant == "budget":
                yield f"reserve {i}", replace(
                    s, held=put(s.held, i, s.cert[i])), None
        if s.held[i] >= 0:
            yield f"cancel reservation {i}", replace(
                s, held=put(s.held, i, -1)), None
        if (current(s, s.held[i], i, mutant)
            and (not s.used[i] or mutant == "replay")):
            for payload in (0, 1):
                if payload != 0 and mutant != "binding":
                    continue
                violation = None
                if not current(s, s.held[i], i, ""):
                    violation = "stale or unauthorised execution"
                if s.used[i]:
                    violation = "action identifier executed twice"
                if payload != 0:
                    violation = "approved payload substituted"
                yield f"execute {i} payload {payload}", replace(
                    s, held=put(s.held, i, -1),
                    used=put(s.used, i, 1), spent=s.spent + 1), violation

def explore(budget, mutant=""):
    start = State()
    queue = deque([start])
    parents = {start: None}
    edge_count = 0
    def path(state):
        result = []
        while parents[state] is not None:
            previous, action = parents[state]
            result.append(action)
            state = previous
        return list(reversed(result))
    while queue:
        s = queue.popleft()
        for action, nxt, violation in transitions(s, budget, mutant):
            edge_count += 1
            if nxt.spent + sum(c >= 0 for c in nxt.held) > budget:
                violation = "aggregate budget exceeded"
            if violation:
                return {"budget": budget, "mutant": mutant or "none",
                    "states_seen": len(parents), "edges_checked": edge_count,
                    "violation": violation, "trace": path(s) + [action]}
            if nxt not in parents:
                parents[nxt] = (s, action)
                queue.append(nxt)
    return {"budget": budget, "mutant": mutant or "none",
        "states_seen": len(parents), "edges_checked": edge_count,
        "violation": None}

```

```

def examples():
    def row(label, us, ud, qs, qd, cs, cd, a=0):
        vs = F(a) + qs * (us - a) - cs
        vd = F(a) + qd * (ud - a) - cd
        return dict(case=label, candidate=str(vs), deviation=str(vd),
                    margin=str(vs-vd), stable_against_this_deviation=vs >= vd)
    return [
        row("optional checking", 3, 3, F(1), F(1), F(1,10), F(0)),
        row("common survival", 3, 1, F(3,10), F(3,10), F(1,10), F(0)),
        row("faster deviation", 3, 1, F(3,10), F(1), F(1,10), F(0)),
        row("positive margin", 3, 1, F(4,5), F(9,10), F(1,10), F(0)),
        row("mandatory evidence routes", 5, 5, F(4,5), F(4,5), F(3,5), F(1,5)),
    ]

if __name__ == "__main__":
    baseline = [explore(b) for b in (1, 2)]
    assert all(x["violation"] is None for x in baseline)
    mutants = [explore(2 if m == "replay" else 1, m)
               for m in ("budget", "epoch", "configuration", "replay", "binding")]
    assert all(x["violation"] is not None for x in mutants)
    result = {"incentive_comparisons": incentive_checks(),
              "baseline": baseline, "mutants": mutants,
              "finite_games": finite_games(), "examples": examples()}
    destination = Path(__file__).with_name("paper1_check_results.json")
    destination.write_text(json.dumps(result, indent=2) + "\n")
    print(json.dumps(result, indent=2))

```

References

- [1] Tennenholtz, M. (2004). Program equilibrium. *Games and Economic Behavior*, 49(2), 363–373. <https://doi.org/10.1016/j.geb.2004.02.002>
- [2] Bárász, M., Christiano, P., Fallenstein, B., Herreshoff, M., LaVictoire, P., and Yudkowsky, E. (2014; revised 2021). Robust Cooperation in the Prisoner’s Dilemma: Program Equilibrium via Provability Logic. arXiv:1401.5577, version 2. Preprint. <https://arxiv.org/abs/1401.5577>
- [3] Critch, A. (2019). A Parametric, Resource-Bounded Generalization of Löb’s Theorem, and a Robust Cooperation Criterion for Open-Source Game Theory. *The Journal of Symbolic Logic*, 84(4), 1368–1381. The theorem numbering used here follows the consulted preprint, Parametric Bounded Löb’s Theorem and Robust Cooperation of Bounded Agents, arXiv:1602.04184, version 5 (2016). <https://doi.org/10.1017/jsl.2017.42> <https://arxiv.org/abs/1602.04184>
- [4] Critch, A., Dennis, M., and Russell, S. (2022). Cooperative and uncooperative institution designs: Surprises and problems in open-source game theory. arXiv:2208.07006. Preprint. <https://arxiv.org/abs/2208.07006>
- [5] Cooper, E., Oesterheld, C., and Conitzer, V. (2025). Characterising Simulation-Based Program Equilibria. *Proceedings of the AAAI Conference on Artificial Intelligence*, 39(13), 13735–13744. <https://doi.org/10.1609/aaai.v39i13.33501>
- [6] Stöcker, C. (2026). Risk-Bounded Runtime Assurance for Dynamic Multi-Agent Systems under Partial Observation. Companion Paper 2, working manuscript, series revision 28 September 2026. Author’s research catalogue: <https://spherity.github.io/spherity-research/>. Companion PDF (same folder).
- [7] Stöcker, C. (2026). Private Verification of Action Eligibility across Trust Domains. Companion Paper 3, working manuscript, series revision 28 September 2026. Author’s research catalogue: <https://spherity.github.io/spherity-research/>. Companion PDF (same folder).
- [8] Fortnow, L. (2009). Program Equilibria and Discounted Computation Time. *Proceedings of the 12th Conference on Theoretical Aspects of Rationality and Knowledge*, 128–133. DOI: 10.1145/1562814.1562833. <https://lance.fortnow.com/papers/files/discount.pdf>
- [9] Halpern, J. Y., and Pass, R. (2008). Game Theory with Costly Computation. arXiv:0809.0024. Consulted preprint. <https://arxiv.org/abs/0809.0024>
- [10] DiGiovanni, A., and Clifton, J. (2023). Commitment Games with Conditional Information Disclosure. *Proceedings of the AAAI Conference on Artificial Intelligence*, 37(5), 5616–5623. <https://doi.org/10.1609/aaai.v37i5.25697>
- [11] Kalai, A. T., Kalai, E., Lehrer, E., and Samet, D. (2010). A commitment folk theorem. *Games and Economic Behavior*, 69, 127–137. <https://doi.org/10.1016/j.geb.2009.09.008>
- [12] Necula, G. C. (1997). Proof-Carrying Code. *Proceedings of the 24th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, 106–119. <https://doi.org/10.1145/263699.263712>
- [13] Chiesa, A., and Tromer, E. (2010). Proof-Carrying Data and Hearsay Arguments from Signature Cards. *Innovations in Computer Science*. Author manuscript. <https://people.csail.mit.edu/alexch/research/pcd/pcd-ics.pdf>

- [14] Saltzer, J. H., and Schroeder, M. D. (1975). The Protection of Information in Computer Systems. *Proceedings of the IEEE*, 63(9), 1278–1308. <https://web.mit.edu/saltzer/www/publications/protection/>
- [15] Xiao-Yang Liu Yanglet, Xiaodong Wang, and Agostino Capponi (2026). No Certificate, No Execution: Certified Traces as a Foundation for Trustworthy AI Agents. arXiv:2605.24462. Preprint, 23 May. <https://arxiv.org/abs/2605.24462>
- [16] Wang, Z. (2026). Proof-Carrying Agent Actions: Model-Agnostic Runtime Governance for Heterogeneous Agent Systems. arXiv:2606.04104. Preprint, 2 June. <https://arxiv.org/abs/2606.04104>
- [17] Hadfield-Menell, D., Dragan, A., Abbeel, P., and Russell, S. (2017). The Off-Switch Game. *Proceedings of the Twenty-Sixth International Joint Conference on Artificial Intelligence*, 220–227. <https://doi.org/10.24963/ijcai.2017/32>
- [18] Herlihy, M. P., and Wing, J. M. (1990). Linearizability: A Correctness Condition for Concurrent Objects. *ACM Transactions on Programming Languages and Systems*, 12(3), 463–492. <https://cs.brown.edu/~mph/HerlihyW90/p463-herlihy.pdf>
- [19] Schneider, F. B. (2000). Enforceable Security Policies. *ACM Transactions on Information and System Security*, 3(1), 30–50. <https://www.cs.cornell.edu/fbs/publications/EnfSecPols.pdf>
- [20] Dalrymple, D., et al. (2024). Towards Guaranteed Safe AI: A Framework for Ensuring Robust and Reliable AI Systems. arXiv:2405.06624. Preprint. <https://arxiv.org/abs/2405.06624>
- [21] Birkholz, H., Thaler, D., Richardson, M., Smith, N., and Pan, W. (2023). Remote Attestation procedureS Architecture. RFC 9334. Informational RFC. <https://www.rfc-editor.org/rfc/rfc9334.html>
- [22] W3C (2025). Verifiable Credentials Data Model v2.0. W3C Recommendation. <https://www.w3.org/TR/vc-data-model-2.0/>
- [23] AI Safety for Mathematicians (no date). Open-source game theory. Research overview and reading list. Accessed 20 September 2026. <https://mathforaisafety.org/research/open-source-game-theory>
- [24] Arun, A., Setty, S., and Thaler, J. (2024). Jolt: SNARKs for Virtual Machines via Lookups. EUROCRYPT 2024. Cryptology ePrint Archive, 2023/1217. <https://www.microsoft.com/en-us/research/publication/jolt-snarks-for-virtual-machines-via-lookups/>
- [25] Cutler, J., et al. (2024). Cedar: A new language for expressive, fast, safe, and analyzable authorization. OOPSLA 2024. <https://www.amazon.science/publications/cedar-a-new-language-for-expressive-fast-safe-and-analyzable-authorization>
- [26] Stöcker, C. (2026). Beyond Single-Enterprise ZTA: Multi-Trust Architectures for Authorised Agentic Actors in Open, Cross-Domain Ecosystems. Research manuscript, 16 September. <https://spherity.github.io/spherity-research/beyond-zero-trust-m-trust-authorised-agent-ic-actors.html>
- [27] Makins, O., Angelini, O., Shams, Z., and Phuong, M. (2026). Multi-Agent AI Control: Distributed Attacks Hamper Per-Instance Monitors. arXiv:2607.07368. Preprint, 8 July. <https://arxiv.org/abs/2607.07368>